

Assignment on Matrix Transpose using Hadoop MapReduce

Please register your group using google form <https://forms.gle/igvLJKSjZm1ctGAk9>. Member 1's roll number will be the **GroupNo**.

By doing this assignment, you are expected to become familiar with docker, understand its advantages and feel confident of using docker for any future projects. You would not only learn the basic usage of Hadoop MapReduce for data parallel computations but also understand a few design considerations for efficiency, especially, the importance of choosing a proper data representation to minimize overheads.

The assignment is split into several parts. Each part has one or more milestones. After completing each milestone, you should attach a proof (screenshot + a short para describing your team's observations / commands / code snippet used to accomplish the task). The proof should be kept in a file named **GroupNo-MILESTONE-x.pdf**, where x is the number of the milestone. The milestone files need to be uploaded as part of the assignment submission. **There should be only 1 submission per group by the student whose rollnumber is the GroupNo.**

Part 1) Getting the environment ready

Useful reference links:

- 1) <https://docs.docker.com/engine/install/ubuntu/>
- 2) <https://towardsdev.com/setting-up-an-hdfs-docker-cluster-a-step-by-step-guide-d4846ff54b5d>
- 3) Useful HDFS and Docker commands required for this assignment
- 4) Slides on Docker and YARN
- 5) Docker cheatsheet.pdf (most of these can be done from the GUI on docker desktop)

A) Setup a Hadoop Cluster using docker compose

Please install the docker engine, and get docker desktop working. Refer to link 2 just to get an understanding of how to setup the hdfs docker cluster. NO need to follow their steps.

We provide you with a folder `hadoop-cluster` that contains the

- 1) required config files and directories for storing data related to the namenode and 2 datanodes,
- 2) helper scripts: `init-datanode.sh`, `start-hdfs.sh`
- 3) `docker-compose.yml`, and
- 4) `input.txt` which needs to be stored in HDFS

Please download the folder `hadoop-cluster.zip` on your machine, unzip it, navigate to that folder, open a terminal there and execute the following command:

`docker compose up`

This would start the `hadoop-cluster` containing a namenode, and 2 datanodes. **MILESTONE 1**

B) Make a directory called `/input` in HDFS. Then store the `input.txt` present in the `hadoop-cluster` folder on `/input` directory in HDFS. Use appropriate hdfs commands for doing this task. Please refer link 3 for relevant commands. **MILESTONE2**

C) For getting the appropriate GUI working e.g. Namenode, Datanode, etc., please update the `docker-compose.yml` exposing relevant ports. **MILESTONE3**

Namenode: port 9870

Datanode: port 9864

D) Submit the wordcount MapReduce job from the namenode. The jar file containing the wordcount program is already provided in `/opt/hadoop/share/hadoop/mapreduce` on

the namenode. To open a shell on the namenode, refer to the docker commands in link 3. Navigate to that folder and execute the MapReduce code as follows:

```
hadoop jar hadoop-mapreduce-examples-3.4.1.jar wordcount /input  
/output
```

The mapreduce program does not run. It says connection refused. Read the error message carefully, and then look at the slides on YARN (*slide 6 has the hint: which component is not running?*) on the google classroom.

Then go through `docker-compose.yml`, `start-hdfs.sh` and `yarn-site.xml`, look for details containing **namenode** and see if you can figure out what is the problem.

All the information required for you to solve the assignment is given in the useful reference links and the slides on google classroom. Please refrain from using google, or any other search engine for this task. Challenge yourself, follow the hints given and see if you can nail down the problem. Please upload your diagnosis of the problem and a possible approach to solve the problem.

MILESTONE4

Part 2: Create input data and store it on HDFS

Write a sequential program to generate 1 GB of input data for the program. Please keep in mind that the input must be a square matrix containing only integers. Please estimate the dimensions accordingly. We have discussed 2 possible formats for storing data: (a) each entry is (i, j, value) , (b) block based storage. In the latter, you create a folder in HDFS and store data for each block in a separate file. Generate the same sized matrix in both these formats, and store them on HDFS. Write down the HDFS storage used for (a) and (b). Attach evidence for it. **MILESTONE5**

Part 3: Write a mapreduce job for matrix transpose and measure its performance

Modify the sample WordCount program (<https://hadoop.apache.org/docs/r3.4.1/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>), compile it into a jar file and then submit on Hadoop MapReduce. You need to write the program only for format (b).

For format (b), you need to read the file name of the block within the mapper. You can use the context object as shown below:

```
import org.apache.hadoop.mapreduce.lib.input.FileSplit;
import org.apache.hadoop.mapreduce.InputSplit;

@Override
protected void map(LongWritable key, Text value, Context context) throws
IOException, InterruptedException {
    InputSplit split = context.getInputSplit();
    if (split instanceof FileSplit) {
        FileSplit fileSplit = (FileSplit) split;
        String fullPath = fileSplit.getPath().toString();
        String fileName = fileSplit.getPath().getName();
    }
    (you can use the filename in your code)
    .
    .
}
```

Verify whether the output of the matrix transpose is correct with a sequential program.
MILESTONE6

Measure performance

Evaluate the performance of the MapReduce job with different sized input matrix, different number of blocks, etc. Explain your observations. **MILESTONE7**